

Projet personnel de collecte de données

Guide de travaux pratiques pour les séances 1 et 2

Vous allez concevoir une petite application de collecte autour d'un sujet personnel, puis construire un pipeline batch capable de transformer des données brutes en un jeu de données fiable et documenté.

Cadre	Valeur
Module	Architectures BD et collecte de données
Durée	Deux séances de 3 heures
Organisation	Individuel ou binôme selon instruction de l'enseignant
Langage conseillé	Python 3.11 ou version ultérieure
Livrable final	Dépôt Git ou archive ZIP reproductible

Mission

Choisissez un domaine qui vous intéresse et imaginez une application ayant besoin de collecter des données pour répondre à une question précise. Votre projet doit rester assez petit pour être démontrable en six heures, mais assez structuré pour montrer une véritable chaîne de collecte.

Résultat attendu

- Séance 1 : cadrage, architecture, sources, schéma, contrat de données et preuve de collecte brute.
- Séance 2 : pipeline exécutable, contrôles de qualité, zones raw et curated, rejets, rapport d'exécution et documentation.

Principe directeur

Une collecte réussie ne se limite pas à obtenir des données. Elle doit être légale, reproductible, compréhensible, contrôlée et utile à une décision.

1 Choisir un projet réaliste

Le sujet est libre. Il doit cependant déboucher sur une application claire, une question mesurable et au moins une source accessible sans contournement technique.

Domaine	Exemple d'application	Données possibles
Mobilité	Comparer des trajets ou stations	Open data, horaires, météo
Environnement	Suivre la qualité de l'air	API publique, mesures locales
Culture	Explorer livres, films ou musées	API ouverte, catalogue public
Sport	Analyser des séances personnelles	Export CSV personnel, données simulées
Énergie	Suivre une consommation	CSV, capteurs simulés, météo
Finance	Observer des indicateurs publics	Open data ou API autorisée
Jeux vidéo	Analyser une collection ou des parties	API publique, historique exporté
Campus	Étudier salles ou équipements	Données ouvertes ou synthétiques

Contraintes obligatoires

- Une question métier ou utilisateur formulée en une phrase.
- Au moins une source structurée CSV, JSON, API ou base de données.
- Au moins 100 observations réelles ou synthétiques cohérentes, sauf exception validée.
- Une zone raw qui conserve une copie non modifiée de la source.
- Une zone curated contenant les données validées et transformées.
- Au moins cinq règles de qualité automatisées.
- Une stratégie explicite de déduplication ou d'idempotence.
- Un rapport d'exécution et un README permettant à un tiers de reproduire le résultat.

Données interdites ou à éviter

- Données personnelles de tiers sans consentement et sans base légale.
- Identifiants, mots de passe, clés API, tokens ou secrets placés dans le dépôt.
- Scraping contraire aux conditions d'utilisation, à robots.txt ou nécessitant un contournement.
- Données médicales, biométriques, scolaires nominatives ou financières sensibles.
- Copie massive d'un site si une API ou un export officiel existe.

Règle de sécurité : en cas de doute, utilisez un jeu de données ouvert ou générez des données synthétiques documentées.

2 Séance 1 Concevoir la collecte

Objectif : transformer une idée personnelle en un plan de collecte précis et produire une première acquisition brute. À la fin de la séance, votre projet doit être cadré et techniquement faisable.

Temps	Étape	Résultat
0 à 20 min	Choix du sujet et question	Pitch de cinq lignes
20 à 50 min	Utilisateur, décision et indicateurs	Trois cas d'usage et trois KPI
50 à 85 min	Inventaire des sources	Fiches source et source principale retenue
85 à 95 min	Pause	
95 à 125 min	Modèle et grain	Schéma de données
125 à 150 min	Architecture et contrat	Diagramme et contrat version 1
150 à 170 min	Première collecte	Fichier horodaté en zone raw
170 à 180 min	Revue de jalon	Validation par un pair ou l'enseignant

Étape 1 Définir l'application

Complétez la fiche suivante avant d'écrire du code.

Question	Réponse attendue
Nom provisoire	Nom court de l'application
Utilisateur	Personne ou rôle qui utilisera le résultat
Problème	Difficulté concrète rencontrée
Décision	Action rendue possible par les données
Question centrale	Question précise à laquelle répondre
Périmètre	Ce qui est inclus et exclu
Fréquence utile	Temps réel, horaire, quotidien ou ponctuel

Étape 2 Définir les cas d'usage et les KPI

Rédigez trois cas d'usage sous la forme suivante :

En tant que [utilisateur],
je veux [action ou information],
afin de [décision ou bénéfice].

Associez ensuite un indicateur à chaque cas d'usage.

Cas d'usage	KPI	Données nécessaires	Fréquence
Exemple : comparer des stations	Disponibilité moyenne	Station, date, capacité	Horaire
Cas 1			
Cas 2			
Cas 3			

Étape 3 Évaluer les sources

Documentez au moins deux sources candidates, même si une seule est finalement utilisée.

Critère	Source A	Source B
Producteur et URL		
Mode d'accès		
Format		
Fréquence de mise à jour		
Clé ou identifiant		
Licence ou conditions		
Risque technique		
Données personnelles		

Critères de sélection

- La source répond-elle directement à la question ?
- L'accès est-il stable, documenté et autorisé ?
- Le volume est-il compatible avec le temps du TP ?
- Les champs possèdent-ils une signification claire ?
- La collecte peut-elle être reproduite par un autre étudiant ?

Étape 4 Définir le grain et le schéma

Écrivez une phrase commençant par Une ligne représente. Cette phrase doit supprimer toute ambiguïté sur l'unité d'observation.

Une ligne représente _____.

Construisez ensuite un dictionnaire initial de cinq à quinze champs.

Champ	Type	Obligatoire	Description	Exemple
observed_at	datetime	Oui	Date et heure de l'observation	2026-09-22T09:00:00Z
entity_id	string	Oui	Identifiant stable de l'entité	STATION-014
value	number	Oui	Mesure principale	17.4
À compléter				

Étape 5 Dessiner l'architecture

Votre diagramme doit faire apparaître au minimum :

- la source et son propriétaire ;
- le collecteur ;
- la zone raw ;
- la future validation ;
- la zone curated ;
- l'application ou la restitution finale.

[Source] -> [Collecteur] -> [Raw] -> [Validation] -> [Curated] -> [Application]

Étape 6 Écrire le contrat de données version 1

```
dataset: nom_du_dataset
version: 1.0
grain: une ligne represente ...
owner: equipe_ou_etudiant
required_fields:
- observed_at
- entity_id
quality_rules:
- observed_at doit etre une date valide
- entity_id ne doit pas etre vide
```

Étape 7 Réaliser la première collecte

La première collecte peut être effectuée par téléchargement documenté ou par script. Elle doit créer une copie horodatée dans la zone raw.

```
project/
data/
raw/
source_20260922T090000Z.json
```

```
curated/  
rejected/  
src/  
reports/  
README.md
```

Jalon de fin de séance 1

Le jalon est validé si vous pouvez montrer les éléments suivants :

- une question centrale non ambiguë ;
- trois cas d'usage et trois KPI ;
- une source principale licite et accessible ;
- un grain et un dictionnaire de données ;
- un diagramme d'architecture ;
- un contrat de données version 1 ;
- un premier fichier raw non modifié.

3 Séance 2 Construire et fiabiliser

Objectif : automatiser la collecte et transformer la donnée brute en un dataset contrôlé. À la fin de la séance, un tiers doit pouvoir relancer votre pipeline et comprendre le résultat.

Temps	Étape	Résultat
0 à 15 min	Revue du jalon 1	Périmètre confirmé
15 à 45 min	Collecteur reproductible	Fichier raw horodaté
45 à 75 min	Profilage	Rapport initial
75 à 90 min	Règles de qualité	Cinq tests minimum
90 à 100 min	Pause	
100 à 135 min	Nettoyage et transformation	Dataset curated
135 à 155 min	Rejets et idempotence	Rejeu sans duplication
155 à 170 min	Rapport et README	Projet reproductible
170 à 180 min	Démonstration	Exécution complète

Étape 1 Construire le collecteur

Le collecteur doit :

- utiliser une URL, un chemin ou un paramètre configurable ;
- gérer explicitement les erreurs d'accès ;
- conserver la réponse brute avant toute transformation ;
- créer un nom de fichier horodaté ;
- ne contenir aucun secret en clair.

```
from datetime import datetime, timezone
from pathlib import Path
import json
import requests

RAW_DIR = Path("data/raw")
RAW_DIR.mkdir(parents=True, exist_ok=True)

def collect(url: str) -> Path:
    response = requests.get(url, timeout=20)
    response.raise_for_status()
    payload = response.json()

    stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
    output = RAW_DIR / f'source_{stamp}.json'
    output.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
    return output
```

Adaptation obligatoire : ce squelette doit être modifié selon votre source. Une copie aveugle sans compréhension ne valide pas le TP.

Étape 2 Profiler les données

Avant de nettoyer, produisez un état initial contenant :

- nombre de lignes et de colonnes ;
- noms et types des champs ;
- taux de valeurs manquantes ;
- nombre de doublons ;
- valeurs minimales et maximales pertinentes ;
- modalités les plus fréquentes pour les champs catégoriels.

```
print(df.shape)
print(df.dtypes)
print(df.isna().mean().sort_values(ascending=False))
print(df.duplicated().sum())
print(df.describe(include="all"))
```

Étape 3 Automatiser cinq règles de qualité

Dimension	Exemple de règle	Action en cas d'échec
Complétude	entity_id est renseigné	Rejeter
Validité	observed_at est une date	Rejeter
Domaine	value est dans une plage plausible	Rejeter ou signaler
Unicité	clé métier non dupliquée	Dédupliquer et compter
Cohérence	start_at est antérieur à end_at	Rejeter
Fraîcheur	donnée reçue dans le délai prévu	Alerter

Chaque règle doit avoir un nom, une condition, un compteur d'échecs et une action.

Étape 4 Séparer les données acceptées et rejetées

```
valid_mask = (  
    df["entity_id"].notna()  
    & df["observed_at"].notna()  
    & df["value"].between(MIN_VALUE, MAX_VALUE)  
)  
  
accepted = df.loc[valid_mask].copy()  
rejected = df.loc[~valid_mask].copy()
```

Ajoutez si possible une colonne indiquant la cause du rejet. Ne supprimez pas silencieusement les lignes invalides.

Étape 5 Transformer et publier

Votre zone curated doit contenir des données :

- typées ;
- normalisées ;
- déduplicuées ;
- compréhensibles sans consulter le code source ;
- alignées sur le contrat de données.

Transformations possibles :

- renommer les colonnes ;
- normaliser les dates en UTC ;
- convertir les unités ;
- harmoniser les catégories ;
- extraire des champs imbriqués ;
- enrichir avec une seconde source ;
- ajouter une variable calculée utile à l'application.

Étape 6 Rendre le pipeline idempotent

Exécutez deux fois le pipeline avec la même entrée. Le résultat final ne doit pas contenir de duplication supplémentaire.

Stratégies acceptées :

- déduplication par clé métier ;
- écriture atomique d'une partition ;
- remplacement contrôlé du fichier de sortie ;
- journal des fichiers déjà traités ;
- UPSERT si une base de données est utilisée.

Étape 7 Produire le rapport d'exécution

```
{  
  "pipeline": "nom-du-projet",  
  "executed_at": "2026-09-22T10:30:00Z",  
  "source_file": "source_20260922T090000Z.json",  
  "input_rows": 1000,  
  "accepted_rows": 962,  
  "rejected_rows": 31,  
}
```

```
"duplicates_removed": 7,  
"quality_status": "PASS_WITH_WARNINGS",  
"duration_seconds": 2.41  
}
```

Étape 8 Documenter et démontrer

Le README doit expliquer :

- l'objectif de l'application ;
- la source et les conditions d'utilisation ;
- les prérequis ;
- l'installation ;
- la commande d'exécution ;
- l'arborescence ;
- les règles de qualité ;
- les limites connues.

```
python -m venv .venv  
# Activation selon le système  
pip install -r requirements.txt  
python src/pipeline.py
```

4 Structure minimale du dépôt

```
personal-data-collector/  
  README.md  
  requirements.txt  
  .gitignore  
  config/  
    data_contract.yaml  
  data/  
    raw/  
      sample_source.json  
    curated/  
      dataset.csv  
    rejected/  
      rejected_rows.csv  
  docs/  
    architecture.png  
    source_assessment.md  
  reports/  
    run_report.json  
  src/  
    collect.py  
    validate.py  
    transform.py  
    pipeline.py  
  tests/  
    test_quality.py
```

Fichier gitignore minimal

```
.venv/  
__pycache__/  
.env  
*.log  
data/raw/*  
!data/raw/sample_source.json
```

Démonstration finale attendue

En trois à cinq minutes, montrez :

1. la question et l'utilisateur cible ;
2. la source et le fichier raw ;
3. une exécution complète ;
4. une donnée volontairement invalide et son rejet ;
5. le dataset curated ;
6. le rapport d'exécution ;
7. une seconde exécution sans duplication ;
8. une limite et une amélioration future.

Critères de non validation

- Aucune source identifiable ou aucune preuve de collecte.
- Dataset final construit manuellement sans pipeline reproductible.
- Absence de règles de qualité automatisées.
- Secrets ou données personnelles exposés dans le dépôt.

- Code impossible à exécuter avec les instructions fournies.
- Données copiées sans respect de la licence ou des conditions d'utilisation.

5 Évaluation

Critère	Points	Preuve attendue
Problème et cas d'usage	3	Question claire, utilisateur, décision et KPI
Sources et conformité	3	Sources documentées, accès licite, limites explicites
Modèle et architecture	3	Grain, dictionnaire, contrat et diagramme cohérents
Collecte reproductible	3	Collecteur configurable et fichier raw horodaté
Qualité et rejets	3	Cinq règles, comptage et conservation des rejets
Transformation et curated	2	Données propres, typées et utiles à l'application
Idempotence et rapport	2	Rejeu contrôlé et rapport d'exécution
Documentation et démonstration	1	README concis et exécution maîtrisée

Niveaux de maîtrise

Niveau	Description
Insuffisant	Le résultat est manuel, incomplet ou impossible à reproduire.
Fragile	Le pipeline fonctionne sur un seul cas mais traite mal les erreurs ou la qualité.
Maîtrisé	La collecte est reproductible, contrôlée, documentée et rejouable.
Avancé	Le projet ajoute tests, paramétrage, seconde source, base de données ou automatisation pertinente.

Bonus possibles

Les bonus améliorent un projet déjà fonctionnel ; ils ne compensent pas l'absence des éléments obligatoires.

- Tests unitaires exécutables.
- Configuration par fichier YAML ou variables d'environnement.
- Chargement dans SQLite ou PostgreSQL avec contrainte d'unicité.
- Collecte de deux sources réellement complémentaires.
- Export Parquet accompagné d'une comparaison de taille ou de temps.
- Planification simple ou workflow d'intégration continue.

6 Checklist de remise

- ☐ Le sujet et la question centrale sont écrits dans le README.
- ☐ La source, la licence et la date d'accès sont documentées.
- ☐ Aucun secret ni donnée personnelle sensible n'est présent.
- ☐ Le dépôt contient un exemple raw de taille raisonnable.
- ☐ Le contrat de données est présent.
- ☐ La commande d'installation fonctionne.
- ☐ La commande d'exécution fonctionne.
- ☐ Les règles de qualité sont automatisées.
- ☐ Les rejets sont conservés et expliqués.
- ☐ Le résultat curated est produit automatiquement.
- ☐ Le pipeline peut être relancé sans duplication incorrecte.
- ☐ Le rapport d'exécution est généré.
- ☐ Le diagramme d'architecture est lisible.
- ☐ Une limite et une amélioration sont discutées.

Questions pour la soutenance

- Quelle décision votre application permet-elle de prendre ?
- Pourquoi cette source plutôt qu'une autre ?
- Quelle est votre clé métier ?
- Quelle règle de qualité a rejeté le plus de données ?
- Que se passe-t-il si la source est indisponible ?
- Que se passe-t-il si le schéma change ?
- Comment savez-vous que le pipeline n'a pas traité deux fois la même donnée ?
- Quelle brique faudrait-il ajouter pour passer en production ?

Fin de l'énoncé